

Ejercitación 1:

1. Qué es HTML, cuando fue creado, cuáles fueron las distintas versiones y cuál es la última?

HTML es la sigla en inglés de HyperText Markup Language, que significa Lenguaje de marcas de hipertexto. Es un lenguaje de marcado estándar que se utiliza para la elaboración y definición de contenido de las páginas web. Fue desarrollado originalmente por Tim Berners-Lee y popularizado por el navegador Mosaic en el año 1993. La versión más actual mencionada en el documento es HTML 5, definida como la quinta revisión más importante que se hace al lenguaje. Aclaración sobre lo que no está en el PDF: El apunte no enumera cuáles fueron las versiones anteriores específicas (como HTML 2, 3 o 4), limitándose a indicar que durante los años 90 proliferó y el lenguaje se desarrolló de diferentes maneras.

2. ¿Cuáles son los principios básicos que el W3C recomienda seguir para la creación de documentos con HTML?

- El W3C (World Wide Web Consortium) es el consorcio internacional dedicado a generar recomendaciones y estándares que aseguren el crecimiento de la web.
- Cualquier tipo de dispositivo debería ser capaz de usar la información de la Web (PCs, teléfonos móviles, dispositivos de voz, computadoras con distintos anchos de banda, etc.).
- Otro principio es la interoperabilidad, es decir, que los documentos HTML deberían funcionar bien en diferentes navegadores y plataformas para que los autores compartan convenciones y reduzcan gastos desarrollando una única versión del documento.

3. En las Especificaciones de HTML, ¿cuándo un elemento o atributo se considera desaprobado? ¿y obsoleto?

- Un elemento o atributo de presentación de HTML se declarará obsoleto en el futuro por el W3C debido a que las hojas de estilo (CSS) ofrecen mejores mecanismos de presentación.
- Mientras tanto, a lo largo de las especificaciones, a dichos elementos y atributos anteriores se los marca como "desaprobados" para advertir sobre su uso.

4. Qué es el DTD y cuáles son los posibles DTDs contemplados en la especificación de HTML 4.01?

La DTD (Document Type Declaration o Declaración del tipo de documento) es una sección que se ubica en la primera línea del archivo HTML, es decir, antes de la marca <html>.

Según el rigor de HTML 4.01 utilizado, los DTDs contemplados son dos:

1. **Declaración transitoria (Transitional):** <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">.

2. **Declaración estricta (Strict):** `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "[http://www.w3.org/TR/html4/strict.dtd](http://www.w3.org/TR/html4/strict.dtd)">`.

Para HTML 5, esto se simplificó a simplemente `<!DOCTYPE HTML>`.

5. Qué son los metadatos y cómo se especifican en HTML?

Los metadatos son información sobre un documento, más que el contenido del documento en sí mismo.

La especificación de metadatos se realiza generalmente siguiendo dos pasos:

1. Declaración de una propiedad y su valor:

Desde dentro del documento: Se utiliza el elemento `<META>`. Este elemento funciona declarando una pareja de propiedad y valor: el atributo `name` identifica el nombre de la propiedad y el atributo `content` le asigna el valor. Por ejemplo: `<META content="Dave Raggett" name="Author">`. Se puede utilizar el atributo `http-equiv` en lugar de `name` cuando se necesita que los servidores HTTP utilicen esa información para crear un encabezado en el mensaje de respuesta (por ejemplo, para indicar la fecha de caducidad del documento).

Desde fuera del documento: Se vinculan los metadatos externos por medio del elemento `<LINK>`.

2. Referencia a un perfil:

Se debe indicar un "perfil" (una especie de diccionario de referencia donde se definen qué propiedades y valores son legales o válidos). Esto se especifica utilizando el atributo `profile` directamente en la etiqueta `<HEAD>` (ej: `<HEAD profile="[http://www.acme.com/profiles/core](http://www.acme.com/profiles/core)">`). Al colocarlo allí, este perfil rige para todos los elementos `<META>` y `<LINK>` que se encuentren en la cabecera.

El elemento `<META>` puede acompañarse de otros atributos para dar más contexto:

- **lang:** Para especificar el idioma en el que está escrito el atributo `content`.
- **scheme:** Para indicar el esquema o formato exacto que se está usando para un dato, evitando ambigüedades (por ejemplo, aclarar si una fecha está en formato Día-Mes-Año o Mes-Día-Año).

Ejercitación 2:

Analizar los siguientes segmentos de código indicando en qué sección del documento HTML se colocan, cuál es el efecto que producen y señalar cada uno de los elementos, etiquetas, y atributos (nombre y valor), aclarando si es obligatorio.

2.a)

`<!-- Código controlado el día 12/08/2009 -->`

- **Sección del documento:** El comentario puede ubicarse en cualquier sección del documento, tanto en el <HEAD> como en el <BODY>.
- **Efecto que produce:** El navegador ignora por completo el contenido encerrado entre <!-- y -->. No genera ningún resultado visual ni funcional en la página. Su único propósito es documentar el código fuente para el desarrollador.
- **Elementos, etiquetas y atributos:**
 - **Elemento / Etiqueta <!-- ... -->:** Es una construcción especial de HTML para comentarios. No es una etiqueta convencional, no tiene contenido renderizable y no admite atributos.

2.b)

```
<div id="bloque1">Contenido del bloque1</div>
```

Sección del documento: Pertenece obligatoriamente al <BODY>, dado que contiene contenido visible destinado al usuario.

Efecto que produce: La etiqueta <div> define un contenedor de bloque genérico. Por sí misma no aplica ningún estilo visual, pero permite agrupar elementos para asignarles propiedades CSS o manipularlos mediante scripts. Al tratarse de un elemento de bloque, genera un salto de línea antes y después de sí mismo.

Elementos, etiquetas y atributos:

- **Elemento / Etiqueta <div>...</div>:** Elemento de bloque genérico. Agrupa contenido sin semántica específica.
- **Atributo id:** Su valor es "bloque1". Es el identificador único del elemento dentro del documento, lo que permite referenciarlo desde CSS o JavaScript. (No es obligatorio).

2.c)

```
<img src="" alt="lugar imagen" id="im1" name="im1" width="32" height="32"
longdesc="detalles.htm" />
```

Sección del documento: Pertenece al <BODY>. Es contenido visual destinado a ser renderizado por el navegador.

Efecto que produce: Inserta una imagen en el flujo del documento en el lugar exacto donde aparece la etiqueta. Al estar el atributo src vacío, la imagen no se cargará. Los atributos width y height reservan el espacio correspondiente aunque la imagen no se muestre.

Elementos, etiquetas y atributos:

- **Elemento / Etiqueta :** Elemento vacío (sin etiqueta de cierre separada). Inserta una imagen en el documento.
- **Atributo src:** Su valor es "". Es la ruta o URL del archivo de imagen a mostrar. Un valor vacío impide la carga de la imagen. (Es obligatorio).

- **Atributo alt:** Su valor es "lugar imagen". Es el texto alternativo. Se muestra si la imagen no carga y es leído por lectores de pantalla. (Obligatorio por accesibilidad en HTML5).
- **Atributo id:** Su valor es "im1". Identificador único del elemento en el documento. (No es obligatorio).
- **Atributo name:** Su valor es "im1". Nombre del elemento. Es un atributo heredado (legacy), reemplazado en la práctica moderna por id. (No es obligatorio).
- **Atributo width:** Su valor es "32". Ancho de la imagen en píxeles. Si se omite, se usa el tamaño original del archivo. (No es obligatorio).
- **Atributo height:** Su valor es "32". Alto de la imagen en píxeles. Si se omite, se usa el tamaño original. (No es obligatorio).
- **Atributo longdesc:** Su valor es "detalles.htm". URL de una página con una descripción detallada de la imagen. Está desaprobado en HTML5. (No es obligatorio).

2.d)

```
<meta name="keywords" lang="es" content="casa, compra, venta, alquiler " /> <meta http-equiv="expires" content="16-Sep-2019 7:49 PM" />
```

Sección del documento: Ambas etiquetas se ubican dentro del <HEAD>. Los metadatos contienen información sobre el documento destinada al navegador y a los motores de búsqueda, y no se muestran al usuario.

Efecto que produce: La primera etiqueta proporciona al motor de búsqueda las palabras clave asociadas al contenido de la página en idioma español. La segunda etiqueta indica al navegador la fecha y hora exacta en que el documento se considera caducado, forzando su recarga desde el servidor en lugar de usar la memoria caché.

Elementos, etiquetas y atributos:

- **Elemento / Etiqueta <meta ... />:** Elemento vacío de metadatos. No genera contenido visible. No es obligatorio incluir metas en una página web.
- **Atributo name (primera etiqueta):** Su valor es "keywords". Define el nombre de la propiedad de metadato que se configura. (No es obligatorio por sí solo).
- **Atributo lang:** Su valor es "es". Especifica el idioma del valor del atributo content. (No es obligatorio).
- **Atributo content (primera etiqueta):** Su valor es "casa, compra, venta...". Asigna el valor a la propiedad indicada en name. (Es obligatorio si usas name o http-equiv).
- **Atributo http-equiv (segunda etiqueta):** Su valor es "expires". Emula un encabezado HTTP. Indica al servidor y al navegador que esta directiva es de tipo protocolo. (No es obligatorio por sí solo).
- **Atributo content (segunda etiqueta):** Su valor es "16-Sep-2019 7:49 PM". Valor asociado a la directiva http-equiv indicando la fecha de caducidad. (Es obligatorio si usas http-equiv).

2.e)

```
<a href="http://www.e-style.com.ar/resumen.html" type="text/html" hreflang="es"
charset="utf-8" rel="help">Resumen HTML </a>
```

Sección del documento: Pertenece al <BODY>. Al ser un enlace interactivo sobre el que el usuario debe hacer clic, forma parte del contenido visible de la página.

Efecto que produce: Genera un hiperenlace que, al ser activado, dirige al usuario hacia el documento externo especificado (puede ser una referencia dentro del mismo documento también). Adicionalmente, comunica al navegador y a los motores de búsqueda información técnica sobre el destino antes de abrirlo.

Elementos, etiquetas y atributos:

- **Elemento / Etiqueta <a>...:** Elemento de anclaje que define un hiperenlace.
- **Contenido:** "Resumen HTML". Es el texto visible del enlace sobre el que el usuario hace clic.
- **Atributo href:** Su valor es "http://www.e-style.com.ar/resumen.html". Es la URL de destino del enlace. (Es obligatorio).
- **Atributo type:** Su valor es "text/html". Informa el tipo MIME del documento de destino. (No es obligatorio).
- **Atributo hreflang:** Su valor es "es". Informa el idioma del documento de destino. (No es obligatorio).
- **Atributo charset:** Su valor es "utf-8". Codificación de caracteres del documento de destino. Está desaprobado en HTML5. (No es obligatorio).
- **Atributo rel:** Su valor es "help". Relación entre el documento actual y el destino, indicando que el enlace es un documento de ayuda. (No es obligatorio).

2.f)

```
<table width="200" summary="Datos correspondientes al ejercicio vencido">
<caption align="top"> Título </caption>
<tr>
<th scope="col">&nbsp;</th>
<th scope="col">A</th>
<th scope="col">B</th>
<th scope="col">C</th>
</tr>
<tr>
<th scope="row">1°</th>
<td>&nbsp;</td>
<td>&nbsp;</td>
<td>&nbsp;</td>
</tr>
<tr>
<th scope="row">2°</th>
<td>&nbsp;</td>
```

```
<td>&nbsp;</td>
<td>&nbsp;</td>
</tr>
</table>
```

Sección del documento: Pertenece al <BODY>. Las tablas son elementos de bloque visuales diseñados para mostrarse al usuario.

Efecto que produce: Genera una tabla de datos con un ancho fijo. Presenta un título centrado en la parte superior y crea una cuadrícula. Las celdas de datos están rellenas con un espacio en blanco () para forzar que sus bordes se dibujen correctamente aunque no haya texto real.

Elementos, etiquetas y atributos:

- **Elemento / Etiqueta <table>...</table>:** Es el elemento raíz que contiene toda la tabla HTML.
- **Atributo width (en table):** Su valor es "200". Ancho de la tabla. Desaprobado en HTML5; se debe usar CSS. (No es obligatorio).
- **Atributo summary (en table):** Su valor es "Datos correspondientes al ejercicio...". Descripción para lectores de pantalla. Desaprobado en HTML5. (No es obligatorio).
- **Elemento / Etiqueta <caption>...</caption>:** Define el título de la tabla.
- **Atributo align (en caption):** Su valor es "top". Posiciona el título arriba. Desaprobado en HTML5. (No es obligatorio).
- **Elemento / Etiqueta <tr>...</tr>:** Define una fila entera de la tabla. (Obligatorio para armar la cuadrícula).
- **Elemento / Etiqueta <th>...</th>:** Celda de encabezado. El navegador la muestra en negrita y centrada por defecto. (No es obligatorio).
- **Atributo scope (en th de columna):** Su valor es "col". Indica que el encabezado aplica a la columna entera. (No es obligatorio).
- **Atributo scope (en th de fila):** Su valor es "row". Indica que el encabezado aplica a la fila entera. (No es obligatorio).
- **Elemento / Etiqueta <td>...</td>:** Celda de dato estándar. (Obligatorio si se van a colocar datos).
- **Contenido :** Entidad HTML que representa un "espacio de no separación". Se usa como truco visual para que las celdas vacías no colapsen. (No es obligatorio).

Ejercitación 3

En cada caso, explicar las diferencias entre los segmentos de código y sus visualizaciones:

3.a)

```
<a href="http://www.google.com.ar">Click aquí para ir a Google</a>
<a href="http://www.google.com.ar" target="_blank">Click aquí para ir a Google</a>
<a href="http://www.google.com.ar" type="text/html" hreflang="es" charset="utf-8"
rel="help">
<a href="#">Click aquí para ir a Google</a>
```

```
<a href="#arriba">Click aquí para volver arriba</a>
<a name="arriba" id="arriba"></a>
```

Los seis segmentos usan la misma etiqueta `<a>` pero con comportamientos distintos:

El primero es un enlace externo básico. Al hacer clic, el navegador navega a la URL indicada en la misma pestaña. El segundo es idéntico en apariencia pero agrega `target="_blank"`, lo que hace que el destino se abra en una pestaña o ventana nueva.

El tercero incorpora atributos adicionales (`type`, `hreflang`, `charset`, `rel`) que describen el documento de destino para el navegador y los motores de búsqueda, sin cambiar la apariencia ni el destino. Cabe notar que en el código este segmento no posee contenido ni etiqueta de cierre visible, por lo que no generaría texto clicable.

El cuarto usa `href="#"`, que es un enlace nulo o placeholder: no navega a ningún destino real, simplemente recarga la página o vuelve al inicio del documento.

El quinto es un enlace interno o ancla: `href="#arriba"` indica que el destino es un punto dentro del mismo documento identificado con `id="arriba"`. Funciona en conjunto con el sexto segmento.

El sexto no es un enlace clicable sino el punto de destino del ancla. Está vacío (`</a>` sin contenido), por lo que es invisible en la página. Su función es marcar la posición exacta del documento hacia donde navega el quinto segmento al hacer clic.

3.b)

```
<p><a href="http://www.google.com.ar">Click
aquí</a></p>
<p><a href="http://www.google.com.ar"></a> Click
aquí</p>
<p><a href="http://www.google.com.ar">Click
aquí</a></p>
<p><a href="http://www.google.com.ar"></a>
<a href="http://www.google.com.ar">Click aquí</a></p>
```

Los cuatro segmentos combinan los mismos elementos (`<img>` y `<a>`) de formas distintas, lo que cambia qué parte es clicable:

En el primero la imagen y el enlace son elementos independientes uno al lado del otro. La imagen no es clicable; únicamente el texto "Click aquí" funciona como enlace.

En el segundo la imagen está envuelta dentro de `<a>`, por lo que sí es clicable. El texto "Click aquí" queda fuera del enlace y no es clicable.

En el tercero tanto la imagen como el texto están contenidos dentro del mismo `<a>`. Ambos son parte de un único enlace clicable: hacer clic en cualquiera de los dos navega al mismo destino.

En el cuarto la imagen y el texto tienen cada uno su propio <a> independiente. Ambos son clicables por separado, pero los dos apuntan a la misma URL.

3.c)

<pre> xxx yyy zzz </pre>	<pre> xxx yyy zzz </pre>	<pre> xxx <li value="2">yyy <li value="3">zzz </pre>	<pre><blockquote> <p>1. xxx
 2. yyy
 3. zzz</p> </blockquote></pre>
--	--	--	---

El primero usa (lista no ordenada): los ítems se muestran con viñetas (•) y sin numeración. No implica secuencia ni jerarquía.

El segundo usa (lista ordenada): el navegador asigna automáticamente números correlativos (1, 2, 3). Semánticamente indica que el orden importa.

El tercero usa tres independientes, cada una con un solo ítem. El atributo value fuerza el número de cada ítem para que la numeración se vea continua (1, 2, 3). Visualmente puede ser similar al segundo, pero estructuralmente son listas separadas sin relación semántica entre sí, lo cual no es la forma correcta de representar una lista ordenada única.

El cuarto no es una lista HTML sino un párrafo <p> dentro de un <blockquote>. La numeración (1., 2., 3.) es texto plano escrito manualmente, sin valor semántico de lista. El <blockquote> agrega indentación al bloque. Es la opción menos correcta semánticamente.

3.d)

<pre><table border="1" width="300"> <tr> <th>Columna 1</th> <th>Columna 2</th> </tr> <tr> <td>Celda 1</td> <td>Celda 2</td> </tr> <tr> <td>Celda 3</td></pre>	<pre><table border="1" width="300"> <tr> <td><div align="center">Columna 1</div></td> <td><div align="center">Columna 2</div></td> </tr> <tr> <td>Celda 1</td> <td>Celda 2</td> </tr> <tr> <td>Celda 3</td></pre>
---	---

<pre><td>Celda 4</td> </tr> </table></pre>	<pre><td>Celda 4</td> </tr> </table></pre>
--	--

El primero usa `<th>` (table header), que es el elemento semántico correcto para los encabezados de columna. El navegador los renderiza automáticamente en negrita y centrados. Los lectores de pantalla los identifican como encabezados y los asocian con las celdas de cada columna, lo que mejora la accesibilidad.

El segundo usa `<td>` (celdas normales) estilizadas manualmente con `<div align="center">` y `<strong>` para imitar visualmente la apariencia de un encabezado. El resultado visual puede ser similar, pero no tiene semántica de encabezado: los lectores de pantalla no los distinguen de las demás celdas de datos. Además, `align` en `<div>` es un atributo desaprobadado en HTML5.

3.e)

<pre><table width="200"> <caption> Título </caption> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd"> &nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> </table></pre>	<pre><table width="200"> <tr> <td colspan="3"><div align="center">Título</div></td> </tr> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> </table></pre>
--	--

El primero usa `<caption>`, que es el elemento semántico específico de HTML para el título de una tabla. El navegador lo posiciona automáticamente encima de la tabla (o debajo, según el CSS), y queda asociado semánticamente a ella. Desde el punto de vista de la accesibilidad, los lectores de pantalla lo anuncian como el título de la tabla antes de leer su contenido. La tabla tiene 2 filas de datos.

El segundo simula un título usando una fila adicional cuya única celda abarca las tres columnas mediante `colspan="3"`, con el texto centrado a través de `<div align="center">`. Visualmente puede ser similar, pero semánticamente es solo una celda más de la tabla, sin ninguna relación formal con ella como título. La tabla tiene 3 filas en total (la primera es el pseudo-título, las otras dos son los datos), lo que altera la estructura real. Además, tanto `bgcolor` como `align` en `<div>` son atributos desaprobadados en HTML5.

3.f)

<pre><table width="200"></pre>	<pre><table width="200"></pre>
--------------------------------------	--------------------------------------

<pre> <tr> <td colspan="3"><div align="center">Título</div></td> </tr> <tr> <td rowspan="2"> bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> </table> </pre>	<pre> <tr> <td colspan="3"><div align="center">Título</div></td> </tr> <tr> <td colspan="2"> bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> <tr> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> <td bgcolor="#dddddd">&nbsp;</td> </tr> </table> </pre>
--	--

Ambos segmentos tienen la misma estructura general: una fila de título que abarca tres columnas seguida de dos filas de datos. La diferencia está en cómo se fusionan las celdas de esas filas.

El primero aplica `rowspan="2"` a la primera celda de la segunda fila. Eso hace que esa celda se extienda verticalmente ocupando las dos filas de datos. Como consecuencia, la tercera fila cuenta con solo dos celdas (la primera columna ya está ocupada por la celda que viene de arriba). El resultado visual es una celda alta en la columna izquierda junto a un bloque de cuatro celdas normales a la derecha.

El segundo aplica `colspan="2"` a la primera celda de la segunda fila. Esa celda se extiende horizontalmente ocupando dos columnas, por lo que esa fila tiene solo dos celdas en lugar de tres. La tercera fila recupera las tres columnas normales. El resultado visual es una celda ancha que ocupa las dos primeras columnas de la segunda fila, con la tercera columna al lado y la tercera fila completamente normal.

3.g)

<pre> <table width="200" border="1"> <tr> <td colspan="3"><div align="center">Título</div></td> </tr> <tr> <td colspan="2" rowspan="2">&nbsp;</td> <td>&nbsp;</td> </tr> <tr> <td>&nbsp;</td> </tr> <tr> <td width="50%">&nbsp;</td> </tr> </table> </pre>	<pre> <table width="200" border="1" cellpadding="0" cellspacing="0"> <tr> <td colspan="2"><div align="center">Título</div></td> </tr> <tr> <td rowspan="2">&nbsp;</td> <td>&nbsp;</td> </tr> <tr> <td width="50%">&nbsp;</td> </tr> </table> </pre>
--	---

La primera diferencia estructural es la cantidad de columnas: el primer segmento tiene 3 columnas (el título abarca las tres con `colspan="3"`), mientras que el segundo tiene 2 (el título abarca las dos con `colspan="2"`).

En cuanto a la fusión de celdas, el primer segmento combina `colspan="2"` y `rowspan="2"` en una misma celda de la segunda fila. Eso genera un bloque rectangular que ocupa simultáneamente 2 columnas y 2 filas, como si se fusionaran cuatro celdas en una sola. En el segundo segmento la fusión es únicamente vertical con `rowspan="2"`: la celda se extiende solo a lo largo de las dos filas, sin abarcar columnas adicionales.

El segundo segmento agrega `cellpadding="0"` y `cellspacing="0"`, que no están en el primero. `cellpadding` controla el espacio interior de cada celda, es decir la distancia entre el borde y el contenido. Con valor 0, el contenido queda pegado al borde sin margen. `cellspacing` controla el espacio entre celdas y entre la tabla y sus bordes exteriores. Con valor 0, los bordes de celdas adyacentes se tocan directamente, eliminando el doble borde que normalmente produce el navegador y logrando un aspecto más compacto.

El atributo `width="50%"` en la última celda del segundo segmento define su ancho como porcentaje del ancho total de la tabla, afectando la distribución del espacio entre las columnas restantes.

3.h)

```
<form id="form1" name="form1" action="procesar.php" method="post" target="_blank">
<fieldset>
<legend>LOGIN</legend>
Usuario: <input type="text" id="usu1" name="usu1" value="xxx" /><br />
Clave: <input type="password" id="clave1" name="clave1" value="xxx" /> </fieldset>
<input type="submit" id="boton1" name="boton1" value="Enviar" />
</form>
```

```
<form id="form2" name="form2" action="" method="get" target="_blank"> LOGIN<br />
<label>Usuario: <input type="text" id="usu2" name="usu2" /></label><br /> <label>Clave:
<input type="text" id="clave2" name="clave2" /></label><br /> <input type="submit"
id="boton2" name="boton2" value="Enviar" />
</form>
```

```
<form id="form3" name="form3" action="mailto:xx@xx.com" enctype=text/plain
method="post" target="_blank">
<fieldset>
<legend>LOGIN</legend>
Usuario: <input type="text" id="usu3" name="usu3" /><br />
Clave: <input type="password" id="clave3" name="clave3" />
</fieldset>
<input type="reset" id="boton3" name="boton3" value="Enviar" />
</form>
```

Los tres presentan un formulario de login con campos de usuario y clave, pero difieren en varios aspectos clave.

Respecto al método de envío, el primero y el tercero usan `method="post"`, que envía los datos en el cuerpo del mensaje HTTP sin exponerlos en la URL, lo cual es más seguro para información sensible. El segundo usa `method="get"`, que anexa los datos directamente a la URL como parámetros visibles, lo que no es adecuado para contraseñas.

Respecto al destino del envío, el primero envía los datos a un archivo del servidor (`action="procesar.php"`). El segundo envía al mismo documento (`action=""` vacío). El tercero usa `action="mailto:xx@xx.com"`, que intenta enviar los datos por correo electrónico a través del cliente de correo configurado en el sistema del usuario; adicionalmente incluye `enctype="text/plain"` para que los datos viajen como texto plano sin codificación especial.

En cuanto a la agrupación visual, los formularios 1 y 3 usan `<fieldset>` con `<legend>` para encuadrar y titular el grupo de campos. El segundo no los utiliza: el título "LOGIN" es simplemente texto suelto seguido de un `<br>`.

Sobre la asociación de etiquetas, el segundo es el único que usa `<label>` envolviendo cada campo, lo que asocia semánticamente el texto descriptivo con su input correspondiente, mejorando la accesibilidad. Los otros dos usan texto suelto junto al input sin asociación formal.

Respecto al campo de clave, los formularios 1 y 3 usan `type="password"`, que oculta los caracteres con asteriscos o puntos al escribir. El segundo usa `type="text"` para ese campo, por lo que la clave se muestra en claro, lo que representa un error tanto de seguridad como de privacidad.

Finalmente, el tipo de botón es distinto en el tercero: mientras los formularios 1 y 2 tienen `type="submit"` que envía el formulario, el tercero tiene `type="reset"`, que borra y reinicia todos los campos sin enviar nada. El hecho de que su value diga "Enviar" es engañoso, ya que el comportamiento real es el opuesto.

3.i)

```
<label>Botón 1
<button type="button" name="boton1" id="boton1">
<br /> <b>CLICK
AQUÍ</b></button></label>
```

```
<label>Botón 2
<input type="button" name="boton2" id="boton2" value="CLICK AQUÍ" />
</label>
```

Ambos generan un botón clicable sin acción predeterminada sobre el formulario (ninguno envía ni limpia por sí solo), pero difieren fundamentalmente en su capacidad de contenido.

El primer segmento usa el elemento `<button>`, que es un elemento contenedor: puede incluir dentro suyo HTML. En este caso contiene una imagen (`<img>`), un salto de línea (`<br>`) y texto en negrita (`<b>`). El botón resultante muestra visualmente la imagen encima del texto "CLICK AQUÍ".

El segundo segmento usa `<input type="button">`, que es un elemento vacío. No puede contener HTML: su único texto posible es el valor del atributo `value`. El botón resultante muestra únicamente el texto plano "CLICK AQUÍ", sin posibilidad de agregar imágenes, formato u otros elementos dentro de él.

En resumen, `<button>` ofrece mucha mayor flexibilidad visual y de contenido, mientras que `<input type="button">` es más simple pero limitado a texto plano.

3.j)

<pre><p><label><input type="radio" name="opcion" id="X" value="X" />X</label>
 <label><input type="radio" name="opcion" id="Y" value="Y" />Y</label></p></pre>
<pre><p><label><input type="radio" name="opcion1" id="X" value="X" />X</label>
 <label><input type="radio" name="opcion2" id="Y" value="Y" />Y</label></p></pre>

Ambos segmentos presentan dos radio buttons con las opciones X e Y, pero su comportamiento difiere completamente según el valor del atributo `name`.

En el primero, ambos inputs tienen `name="opcion"`. El atributo `name` es el que define a qué grupo pertenece cada radio button: cuando dos o más comparten el mismo `name`, el navegador los trata como un grupo de selección exclusiva. Esto significa que solo se puede tener uno seleccionado a la vez. Al elegir uno, el otro se deselecciona automáticamente. Es el comportamiento esperado para un radio button. Al enviar el formulario, se transmite un único par clave-valor: `opcion=X` o `opcion=Y`, según cuál esté seleccionado.

En el segundo, cada input tiene un `name` diferente (`opcion1` y `opcion2`). Al no compartir `name`, el navegador los considera controles independientes y pertenecientes a grupos distintos. Ambos pueden estar seleccionados al mismo tiempo, perdiendo el comportamiento exclusivo que es la razón de ser de los radio buttons. Al enviar el formulario, se transmiten dos pares de clave-valor simultáneamente: `opcion1=X` y `opcion2=Y`. Funcionan más como checkboxes, no como radio buttons.

3.k)

<pre><select name="lista"> <optgroup label="Caso 1"> <option>Mayo</option> <option>Junio</option> </optgroup> <optgroup label="Caso 2"> <option>Mayo</option> <option>Junio</option> </optgroup> </select></pre>	<pre><select name="lista[]" multiple="multiple"> <optgroup label=" Caso 1"> <option>Mayo</option> <option>Junio</option> </optgroup> <optgroup label=" Caso 2"> <option>Mayo</option> <option>Junio</option> </optgroup> </select></pre>
--	--

Los dos segmentos comparten la misma estructura interna de opciones agrupadas con `<optgroup>`, pero difieren en la cantidad de selecciones permitidas y en su presentación visual.

El primero es un `<select>` simple. Se muestra como un menú desplegable (dropdown): el usuario ve únicamente la opción actualmente seleccionada y debe hacer clic para desplegar la lista. Solo puede elegir una opción a la vez. Al enviar el formulario se transmite un único valor bajo la clave lista.

El segundo agrega el atributo `multiple="multiple"`, que transforma el control en una lista visible y expandida donde todas las opciones se muestran simultáneamente sin necesidad de desplegar nada. El usuario puede seleccionar varias opciones a la vez manteniendo presionada la tecla Ctrl o Shift al hacer clic. Otra diferencia es el atributo `name="lista[]"`: la notación de corchetes es una convención utilizada principalmente en lenguajes del lado del servidor como PHP para indicar que el campo puede enviar múltiples valores, los cuales serán recibidos como un array. Sin esa notación, al seleccionar varias opciones el servidor solo recibiría la última.

En ambos casos el elemento `<optgroup>` funciona de la misma manera: agrupa visualmente las opciones bajo una etiqueta (Caso 1, Caso 2) que no es seleccionable por el usuario, solo sirve como encabezado organizativo dentro de la lista.